Projet de Recherche d'Information (Partie 1/3) :
Interface Administrateur d'Indexation Dynamique

Niveau : Ingénierie en Intelligence Artificielle (Semestre 9)
Module : Recherche d'Information (RI)
Durée allouée : 4 Semaines
Organisation : Groupes de 2 étudiants maximum

1. Contexte et Objectif Global du Projet

L'objectif de ce projet semestriel est de concevoir un moteur de recherche intelligent de bout en bout. Le projet est divisé en deux grandes phases :

Phase 1 (Ce TP - 4 semaines) : Création du **Module Administrateur**. L'administrateur système prépare les données, configure les chaînes de traitement linguistique (NLP), génère les index mathématiques et les sauvegarde.

Phase 2 (Prochain TP) : Création du **Module Client/Guest**. L'utilisateur final effectue des recherches via des requêtes, le système calcule la similarité (Matching) et affiche les K meilleurs résultats.

2. Description de la Phase 1 (Module Administrateur)

Dans cette première partie, vous devez développer une interface graphique (GUI ou Web App) destinée exclusivement à l'administrateur du moteur de recherche.
L'interface doit permettre d'ingérer un sous-ensemble du célèbre corpus **Reuters-21578**, d'appliquer un pipeline de prétraitement hautement paramétrable, de générer de multiples structures d'indexation, de mesurer leurs performances en mémoire, et de les sauvegarder sur le disque.

2.1. Le Jeu de Données (Dataset)

Source : Reuters-21578 Text Categorization Collection.
Lien 1 avec Tags: UCI Machine Learning Repository
Lien 3 sans Tags: https://www.kaggle.com/datasets/narendrageek/reuters21578-multilabel-classification-news/data

2.2. Spécifications Fonctionnelles de l'Interface

L'interface administrateur doit obligatoirement inclure les fonctionnalités suivantes :

a) **Sélection des Données :** Un bouton/explorateur pour charger les documents depuis le disque local.
b) **Pipeline NLP Dynamique :** L'administrateur doit pouvoir cocher les étapes de prétraitement qu'il souhaite appliquer ET **définir leur ordre d'exécution**.

Les opérations exigées sont :

- *Tag Cleaning* (Nettoyage des balises SGML/XML du corpus Reuters).
- *Normalisation* (Minuscules, suppression de la ponctuation, etc.).
- *Stopwords Removal* (Suppression des mots vides).
- *Stemming* (Racinisation).
- *Lemmatization* (Lemmatisation).

c) **Lancement de l'Indexation :** Un bouton "Générer les Index" qui déclenche le traitement.
d) **Génération Multi-Modèles :** Le système doit générer, en arrière-plan, TOUS les modèles suivants dans deux formats différents (Matrice et Liste) :

- **BoW (Bag of Words / Binaire) :** Présence (1) ou Absence (0).
- **F (Fréquence) :** Comptage brut des occurrences.

- **TF (Term Frequency) :** Fréquence normalisée (ex: tf / max_tf).
- **TF-IDF :** Poids pondéré global ($TF \times IDF$).
- ***Formats output exigés :*** Matrice Document-Terme (Lignes=Mots, Colonnes=Docs), Matrice Terme- Document (Lignes= Docs, Colonnes= Mots) ET Liste Inversée (Posting Lists) de chaque modèle.

e) **Monitoring des Données (Output) :** L'interface doit afficher des métriques post-indexation :

- La taille du vocabulaire généré (dimension).
- Le nombre de documents indexés.
- La taille occupée en mémoire (RAM) par chaque type d'index (Matrice vs Liste Inversée).

f) **Sérialisation (Sauvegarde) :** Une option permettant à l'administrateur de sauvegarder les index générés sur le disque dur local sous forme de fichiers (binaires, texte ou JSON), afin qu'ils puissent être chargés plus tard par le module de recherche (Phase 2) sans avoir à tout recalculer.

3. Choix Technologiques et Contraintes de Programmation

- **Liberté Totale :** Vous êtes entièrement libres de choisir le langage de programmation (Python, Java, C++, JavaScript, etc.) et les bibliothèques d'interface graphique de votre choix.
- **Programmation "From Scratch" (Forte Recommandation) :** Il y a une forte préférence pour la programmation manuelle des fonctionnalités. L'utilisation d'API ou de bibliothèques haut niveau (comme NLTK, spaCy, ou même le module re pour les expressions régulières complexes) est fortement déconseillée. Vous êtes encouragés à implémenter vous-mêmes les algorithmes (ex: manipulation directe des chaînes de caractères pour le nettoyage, utilisation de dictionnaires manuels pour les mots-vides, algorithmes de stemming basiques codés à la main).

4. Critères d'Évaluation (Livrables et Notation)

L'évaluation de ce projet sera stricte et se basera sur les éléments suivants :

- **Le Code Source :** L'ensemble des scripts et fichiers du projet.
- **Un Rapport Détaillé (Format PDF/Word) :** Ce rapport est obligatoire et doit contenir :
    - Une description générale de l'architecture du projet réalisé.
    - Des **imprimés écrans (screenshots)** de l'interface, accompagnés d'explications claires sur les modules et fonctionnalités réalisés.
    - Une section dédiée aux fonctionnalités **non réalisées** (ce qui manque par rapport au cahier des charges) avec des justifications.
    - Une section détaillant les **difficultés majeures rencontrées** lors du développement et comment vous avez tenté de les surmonter.
- **Consultation en Présentiel (Maîtrise du Code) :** Une note spécifique sera attribuée lors d'une séance de consultation en face-à-face. Chaque étudiant devra démontrer sa compréhension de l'architecture et prouver qu'il maîtrise le code écrit (1 question chacun sur des fonctions précises, modifications en direct, etc.).
- **Note Bonus / Qualité "From Scratch" (Minimisation des API) :** Une note ou bonification sera explicitement attribuée aux groupes ayant programmé les fonctionnalités complexes (notamment le NLP) en utilisant le **minimum d'API et de bibliothèques externes**. L'effort algorithmique "manuel" sera fortement récompensé (effort vérifié par la question face a face).

5. Préparation et Anticipation de la Phase 2 (Pour aller plus loin)

Afin de vous aider à concevoir la meilleure architecture logicielle possible dès le TP1, voici un aperçu de ce qui vous sera demandé lors du prochain projet (Phase 2). Les groupes

qui terminent la Phase 1 en avance sont encouragés à entamer ces développements :

- **Interface de Recherche (Côté Client/Guest) :** Vous devrez créer une seconde interface accessible aux utilisateurs non authentifiés (Guest/Client). Cette interface leur permettra de saisir une requête et d'afficher les meilleurs résultats.
- **Fonction de Matching et Paramètres :** Le module de recherche devra traiter la requête utilisateur, calculer la similarité cosinus avec l'index chargé depuis le disque, et renvoyer les documents pertinents.
- **Rôle Administrateur (Suite) :** L'administrateur aura accès à de nouveaux paramètres de configuration de recherche :
    - Définir la **taille maximale autorisée pour une requête** (nombre de mots).
    - Sélectionner la **fonction de matching** à utiliser (Produit scalaire, Cosinus, etc.).
    - Définir la variable **K** (nombre de meilleurs résultats affichés au client, ex: Top 5, Top 10).
- **Conseil architectural :** Pensez votre code de manière modulaire (Architecture MVC) pour séparer facilement la logique d'indexation (Phase 1) de la logique de recherche (Phase 2), tout en utilisant les mêmes fonctions de prétraitement NLP pour les requêtes utilisateur.